

Тема 4. Абстрактни класове и интерфейси

Заместване на полета при наследяване

При наследяване е възможно името на поле в производния клас да съвпада с името на полето, декларирано в суперкласа. За да се различават двете полета, тогава когато се обръщаме към полето на суперкласа се поставя ключовата дума **super** и се използва точков синтаксис.

Пример 1: Заместване на полета при наследяване

```
//Суперклас:
class Alpha{
int number;
}
//Производен клас:
class Beta extends Alpha{
int number;

void set(int m,int n){
    super.number=m;
    number=n;
}

void show(){
    System.out.println("Alpha: "+super.number);
    System.out.println("Beta: "+number);
}
}
//Главен клас:
class test1 {
public static void main(String[] args){
    Beta obj=new Beta();
    obj.set(1,2);
    obj.show();
    obj.number=3;
    obj.show();
}
}
```

Изход в конзолата:

Alpha: 1

Beta: 2

Alpha: 1

Beta: 3

Многослойно наследяване

Многослойно наследяване – вече съществуващ производен клас се използва като суперклас за създаване на новия производен клас

Пример 2: Многослойно наследяване

```
//Суперклас:
class Alpha1 {
int number1;
Alpha1(int a){
    number1=a;
    System.out.println("Поле на метод Alpha1: "+number1);
}
}

//Производен на класа Alpha1:
class Beta1 extends Alpha1 {
int number2;
Beta1(int a,int b){
    super(a);
    number2=b;
    System.out.println("Поле на метод Beta1: "+number2);
}
}

//Производен на класа Beta1:
class Gamma1 extends Beta1 {
int number3;
Gamma1(int a,int b,int c){
    super(a,b);
    number3=c;
    System.out.println("Поле на метод Gamma1: "+number3);
}
}

//Главен клас:
class test2 {
public static void main(String[] args){
    Gamma1 obj=new Gamma1(1,2,3);
}
}
```

Изход в конзолата:

Поле на метод Alpha1: 1

Поле на метод Beta1: 2

Поле на метод Gamma1: 3

Презареждане и предефиниране на методи

Пример 3: Презареждане и предефиниране на методи

```
//Суперклас:
class Alpha2{
void show(){
    System.out.println("Метод на класа Alpha2");
}
}

//Производен клас на класа Alpha2:
class Beta2 extends Alpha2{
void show(String text){
    System.out.println(text);
}
}

//Производен клас на класа Beta2:
class Gamma2 extends Beta2{
void show(){
    System.out.println("Метод на класа Gamma2");
}
}

//Главен клас:
class test3{
public static void main(String[] args){
    Alpha2 A=new Alpha2();
    Beta2 B=new Beta2();
    Gamma2 C=new Gamma2();
    A.show();
    B.show();
    B.show("Клас Beta2");
    C.show();
    C.show("Клас Gamma2");
}
}
```

Изход в конзолата:

Метод на класа Alpha2

Метод на класа Alpha2

Клас Beta2

Метод на класа Gamma2

Клас Gamma2

Обектни променливи на суперкласа

Обектните променливи на суперкласа могат да се отнасят към обекти на производния клас, но само към онези членове на производния клас, които са били наследени от суперкласа (като наследените членове могат да бъдат предефинирани в производния клас).

Пример 4: Обектна променлива на суперкласа

```
//Суперклас:
class Alpha3{
char symbol;
void set(char s){
    symbol=s;
}
void show(){
    System.out.println("Клас Alpha3");
    System.out.println("Символ: "+symbol);
}
}

//Производен клас:
class Beta3 extends Alpha3{
int number;
void set(char s,int n){
    symbol=s;
    number=n;
}
void show(){
    System.out.println("Клас Beta3");
    System.out.println("Символ: "+symbol);
    System.out.println("Число: "+number);
}
}

//Главен клас:
class test4{
public static void main(String[] args){
    Alpha3 A;
    Beta3 B=new Beta3();
    A=B;
    B.set('B',1);
    B.show();
    A.set('A');
    A.show();
}
}
```

Платформено независими програмни езици

Семинарно упражнение 4

Изход в конзолата:

Клас Beta3

Символ: B

Число: 1

Клас Beta3

Символ: A

Число: 1

Обектните променливи A и B се отнасят към един и същи обект! Чрез обектната променлива A версията на метод **set()** с два аргумента не може да бъде извикана, защото тя не е описана в суперкласа A. По аналогичен начин, чрез обектната променлива A можем да се обърнем към полето **symbol** на обекта на производния клас, но не и към полето **number**.

При извикване на метода версията му се определя не от типа на обектната променлива, чрез която е извикан, а от типа на обекта, към който се отнася променливата!

Пример 5: Обектна променлива

//Суперклас:

```
class Alpha4{
void show(){
    System.out.println("Клас Alpha4");
}
}
```

//Производен клас на класа Alpha4:

```
class Beta4 extends Alpha4{
void show(){
    System.out.println("Клас Beta4");
}
}
```

//Производен клас на класа Alpha4:

```
class Gamma4 extends Alpha4{
void show(){
    System.out.println("Клас Gamma4");
}
}
```

//Главен клас:

```
class test5{
public static void main(String[] args){
    Alpha4 A=new Alpha4();
    Beta4 B=new Beta4();
    Gamma4 C=new Gamma4();
    Alpha4 P;
    P=A;
    P.show();
    P=B;
}
```

```
P.show();  
P=C;  
P.show();  
}  
}
```

Изход в конзолата:

Клас Alpha4

Клас Beta4

Клас Gamma4

Абстрактни класове и методи

Абстрактен метод – метод, чието тяло не е дефинирано (т.е. методът не е имплементиран!). Декларира се само сигнатурата. Използва се ключова дума **abstract**.

Абстрактен клас – клас, който съдържа поне един абстрактен метод. Дефинирането му започва с ключовата дума **abstract**. Абстрактният клас не може да се използва за създаване на обекти. Тяхното основно приложение е да бъдат суперкласове на базата, на които да се създадат производни класове. При тази ситуация абстрактният метод трябва да бъде имплементиран (определен) в явен вид. В противен случай, производният клас също ще бъде абстрактен клас. Декларирането на обектни променливи от абстрактен клас е възможно. Такива променливи могат да се отнасят към обекти на производния клас, наследяващи абстрактния суперклас.

Приложение: Абстрактните класове се използват като шаблони, които определят какво трябва да има задължително в производния клас. Конкретната реализация (имплементация) се прави в производния клас. Тази функционалност облекчава многократното предефиниране на методи и тяхното проследяване.

Пример 6: Абстрактен клас

```
//Абстрактен суперклас:  
abstract class Alpha5 {  
abstract void method1();  
void method2() {  
    System.out.println("Втори метод");  
}  
}  
  
//Производен клас:  
class Beta5 extends Alpha5 {  
void method1() {  
    System.out.println("Първи метод");  
}  
}
```

//Главен клас:

```
class test6{  
public static void main(String[] args){  
    Beta5 B=new Beta5();  
    B.method1();  
    B.method2();  
    Alpha5 A;  
    A=B;  
    A.method1();  
    A.method2();  
}  
}
```

Изход в конзолата:

Първи метод

Втори метод

Първи метод

Втори метод

Анонимни класове и методи

Анонимният клас се създава чрез командата за създаване на обекта на негова основа (това всъщност е обектът, който се създава на база на анонимен клас). Потребността от такива обекти имаме в ситуации, когато обектът се използва само веднъж на едно място в програмата. Анонимен е клас, който няма име. Анонимният клас се създава чрез наследяване на суперклас (абстрактен (най-често) или обикновен). Затова референция към създадения обект може да бъде записана в обектна променлива на абстрактния суперклас.

Създаване на обект на база анонимен клас

```
суперклас променлива = new суперклас(аргументи){  
    // дефиниране на методи (абстрактните методи в суперкласа, както и  
    // предефиниране на обикновени методи от суперкласа)  
}
```

Пример 7: Анонимен клас на база абстрактен суперклас

//Абстрактен клас:

```
abstract class MyClass{  
int number;  
MyClass(int n){  
    number=n;  
}  
  
abstract void show();
```

Платформено независими програмни езици
Семинарно упражнение 4

```
}  
//Главен клас:  
class test7{  
public static void main(String[] args){  
    // Обектът се създава на основата на анонимен клас:  
    MyClass A=new MyClass(10){  
        void show(){  
            System.out.println("Обект A: "+number);  
        }  
    };  
    // Обектът се създава на основата на анонимен клас:  
    MyClass B=new MyClass(20){  
        void show(){  
            System.out.println("Обект B: "+number);  
        }  
    };  
    // Извикване на метода:  
    A.show();  
    B.show();  
    // Промяна на стойностите на полетата:  
    A.number=15;  
    B.number=25;  
    // Извикване на метода:  
    A.show();  
    B.show();  
}  
}
```

Изход в конзолата:

Обект A: 10

Обект B: 20

Обект A: 15

Обект B: 25

Важно! Обектните променливи *A* и *B* се отнасят към различни анонимни класове, макар и да имат еднакви полета и методи (методите са дефинирани по различен начин). И двата класа са анонимни и са производни класове на суперкласа *MyClass*.

Интерфейси

Интерфейсите са средство за реализиране на ползите от множественото наследяване (повишаване на гъвкавостта на програмите).

Интерфейсите съдържат сигнатури на методи (без самата дефиниция) и полета константи (стойностите са постоянни и не могат да се променят), т.е. приличат на абстрактните класове. В последните версии на JAVA, методите могат да бъдат и дефинирани. Тогава в сигнатурата на метода се използва ключовата дума **default**. Всички методи (декларирани и/или дефинирани) в интерфейса, автоматично ще считат за публични, т.е. със спецификатор за достъп **public**, макар и да не се указва. Декларираните и дефинираните методи могат да бъдат статични или частни. Полетата, дефинирани в интерфейса по подразбиране се считат за статични и константи, но ключовите думи **final** и **static** не се пишат. За да се използва даден интерфейс, то той трябва да бъде имплементиран в клас. Ако клас имплементира интерфейс, това означава, че в него трябва да бъдат дефинирани всички методи от интерфейса (използва се ключовата дума **public**). Ако остане недефиниран метод, то класът става абстрактен и трябва да се използва ключовата дума **abstract**. Ако даден метод е вече дефиниран, може и да не се дефинира в класа като в тази ситуация ще се използва кода по подразбиране от интерфейса. Един и същ клас може да имплементира едновременно няколко интерфейса! Един и същи интерфейс може да бъде имплементиран от няколко класа.

Деклариране на интерфейс

```
interface име {  
    // деклариране на полета и методи  
}
```

Деклариране на клас, който имплементира интерфейс

```
class име implements интерфейс {  
    // дефиниране на класа  
}
```

Деклариране на клас, който имплементира няколко интерфейса

```
class име implements интерфейс1, интерфейс2, интерфейс3 {  
    // дефиниране на класа  
}
```

Имената на интерфейсите се разделят със запетая!

Деклариране на клас, който имплементира интерфейс и наследява суперклас

```
class име extends суперклас implements интерфейси {  
    // дефиниране на класа  
}
```

Пример 8: Клас, който имплементира интерфейс

```
//Интерфейс:  
interface Alpha8 {  
int NUMBER=2;  
void setValue(int n);  
String getValue();  
default void show(){  
    System.out.println("Резултат: "+getValue());  
}  
}  
  
//Клас, имплементиращ интерфейс Alpha:  
class MyClass8 implements Alpha8 {  
private int num;  
    // Дефиниране на методите от интерфейса.  
    // Метод за присвояване на стойност на полето:  
public void setValue(int n) {  
    if(n>=0) num=n;  
    else num=-n;  
    System.out.println("Число: "+num);  
}  
    // Метод за преобразуване на стойността на полето в двоично число:  
public String getValue() {  
    String res="";  
    int num1=num;  
    do {  
        res="|"+(num1%NUMBER)+res;  
        num1/=NUMBER;  
    } while(num1>0);  
    return res;  
}  
}  
  
//Главен клас:  
class test8 {  
public static void main(String[] args) {  
    // Създаване на обект:  
    MyClass8 obj=new MyClass8();  
    // Показване на стойността на полето:
```

```
System.out.println("NUMBER: "+Alpha8.NUMBER);  
// Извикване на методите от обекта:  
obj.setValue(15);  
obj.show();  
}  
}
```

Изход в конзолата:

NUMBER: 2

Число: 15

Резултат: |1|1|1|1|

Пример 9: Клас, който имплементира два интерфейса

```
//Първи интерфейс:  
interface First{  
void hello();  
}  
//Втори интерфейс:  
interface Second{  
void hola();  
}  
//Класът имплементира два интерфейса:  
class MyClass9 implements First, Second{  
// Дефиниране на метода от първия интерфейс:  
public void hello(){  
    System.out.println("Метод от интерфейс First");  
}  
// Дефиниране на метода от втория интерфейс:  
public void hola(){  
    System.out.println("Метод от интерфейс Second");  
}  
}  
//Главен клас:  
class test9{  
public static void main(String[] args){  
    // Създаване на обект:  
    MyClass9 obj=new MyClass9();  
    // Извикване на методите от обекта:  
    obj.hello();  
    obj.hola();  
}  
}
```

Изход в конзолата:

Метод от интерфейс First

Метод от интерфейс Second

Интерфейсни променливи

На основата на интерфейси не могат да се създават обекти, но може да бъде декларирана интерфейсна променлива. Тя ще се отнася към обект на клас, в който е имплементиран интерфейса, но ще има достъп само до декларираните в интерфейса методи.

Пример 10: Интерфейсна променлива

```
//Интерфейс:
interface Myinterface {
void show();
}

//Класът имплементира интерфейса:
class Alpha10 implements Myinterface{
String word;

Alpha10(String text){
    word=text;
}

public void show(){
    System.out.println("Обект на клас Alpha10");
    System.out.println("Текстово поле: "+word);
}
}

//Класът имплементира интерфейса:
class Beta10 implements Myinterface{
int number;

Beta10(int n){
    number=n;
}

public void show(){
    System.out.println("Обект на клас Beta10");
    System.out.println("Целочислено поле: "+number);
}
}

//Главен клас:
class test10{
public static void main(String[] args){
    // Интерфейсна променлива:
    Myinterface R;

    R=new Alpha10("BRAVO");
```

Платформено независими програмни езици
Семинарно упражнение 4

```
R.show();  
  
R=new Beta10(6);  
R.show();  
}  
}
```

Изход в конзолата:

Обект на клас Alpha10

Текстово поле: BRAVO

Обект на клас Beta10

Целочислено поле: 6

Задачи за самостоятелна работа:

Задача 1: Дефинирайте абстрактен клас Figure, съдържащ неимплементирани методи за изчисляване на лицето на 2D фигури и принтиране в конзолата на името на фигурата и нейния цвят. Предвидете полета за цвета на фигурата и стойността на линейния размер на фигурата. Дефинирайте производни класове за триъгълник, правоъгълник, квадрат и окръжност, които го наследяват и имплементират абстрактните методи. Използвайте конструктори.